BIASHARA OS

From first deployment to verified MinIO invoice upload

Chronological Kubernetes homelab build, configuration, troubleshooting, and validation. Commands annotated with the machine that runs them and why each step exists.

Evidence labels: Recorded means the command or output appears in the deployment conversation or earlier documented artifacts. **Reconstructed setup example** means it is a reproducible starting point based on the known finished topology, **not** a claim that the exact command was typed previously. The original first-install terminal transcript is not fully available.

Security: No real tokens or secrets are printed. Do not run installation/init/delete examples against your already-running cluster; they are for documentation or a fresh rebuild.

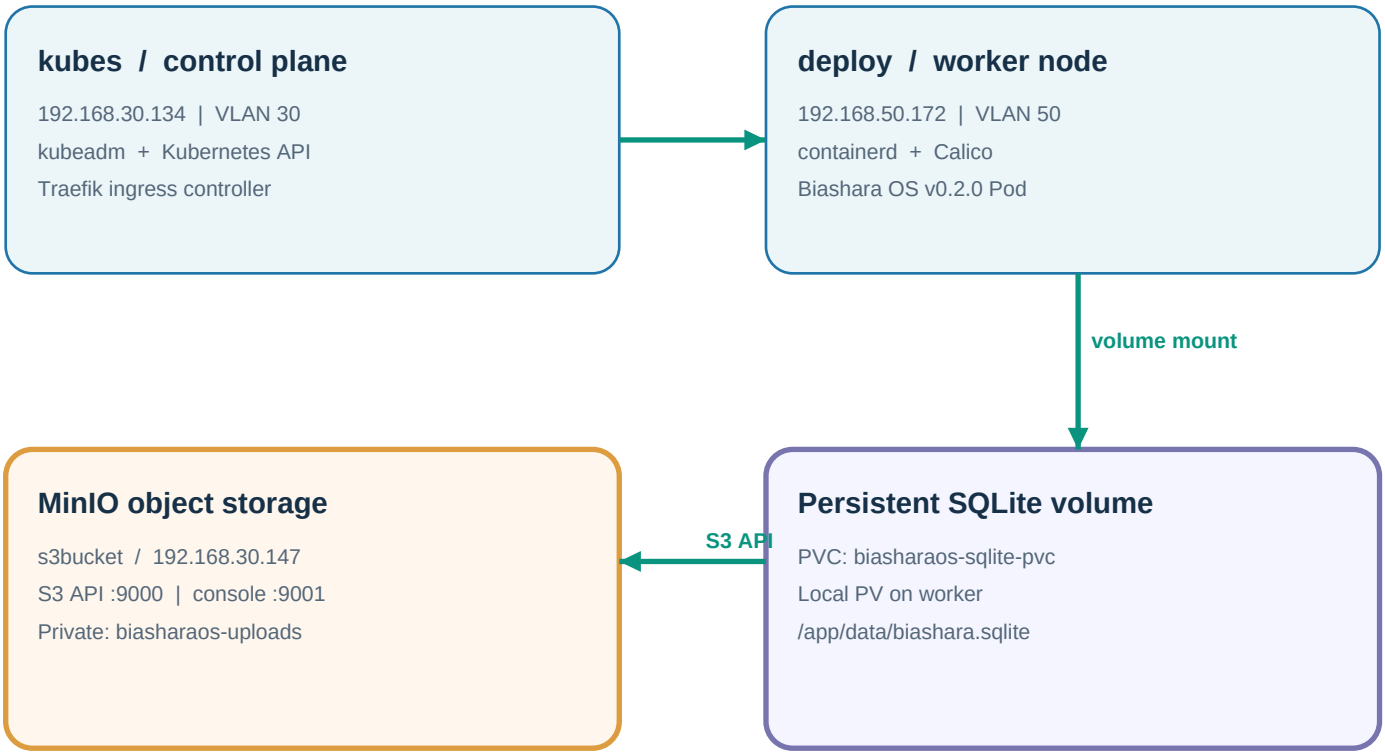
The environment that was built

Role / VM	Network	Responsibility
kubes (Ubuntu 24.04)	192.168.30.134 / VLAN 30	kubeadm control plane; Traefik; kubectl
deploy (Ubuntu 24.04)	192.168.50.172 / VLAN 50	Kubernetes worker; Node.js Pod; local SQLite PV
s3bucket (Debian LXC)	192.168.30.147 / VLAN 30	MinIO S3 API :9000; console :9001
Pod network	10.244.0.0/16	Calico IPIP encapsulation and egress NAT

Target result: Node.js 24 application in namespace **biasharaos**, one replica with **Recreate** rollout strategy, local SQLite database under **/app/data/biashara.sqlite**, and private MinIO bucket **biasharaos-uploads** holding test PDF invoices.

Homelab infrastructure topology

Logical layout: control plane, worker, application data and object storage.



Cluster networking

Calico Pod CIDR: 10.244.0.0/16

Pod-to-MinIO access depends on working routing and outbound NAT

Why separate SQLite and invoices?

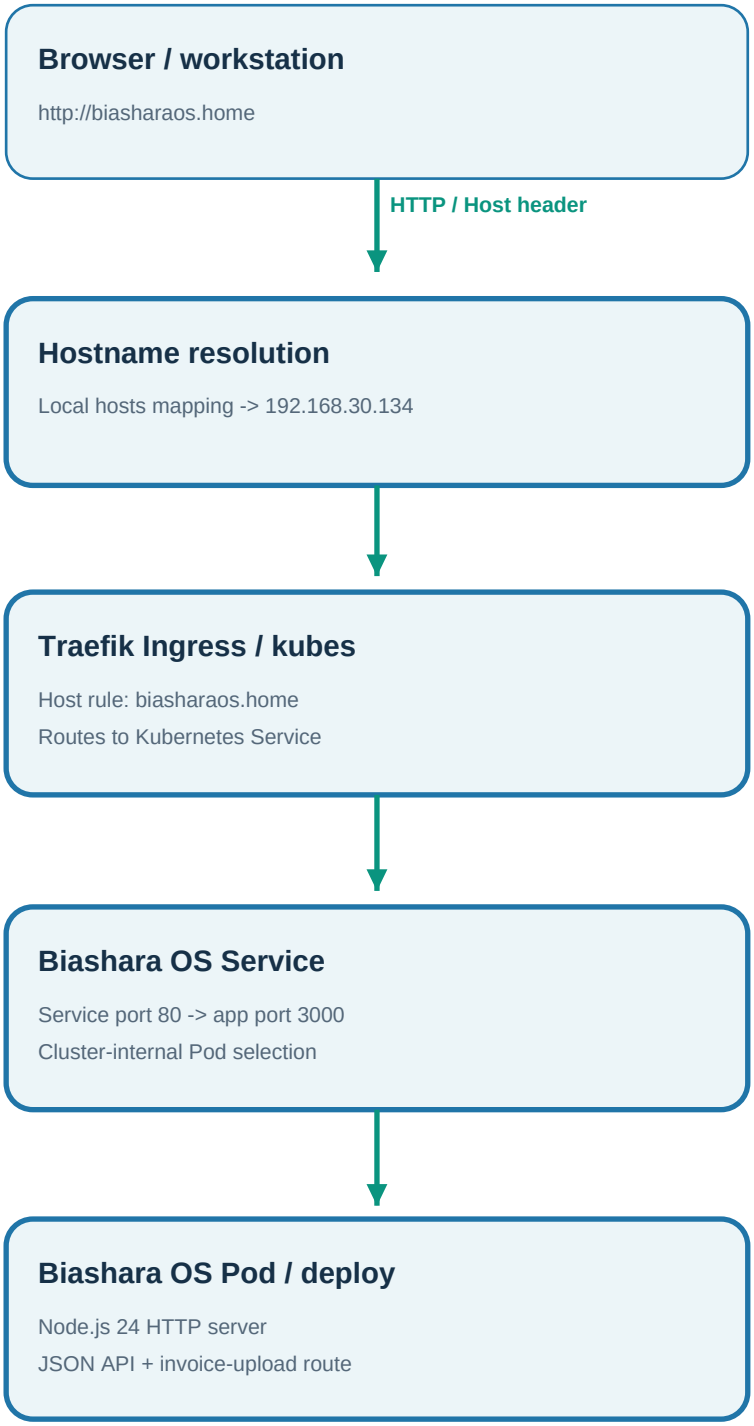
SQLite remains on the worker persistent volume for business records. Uploaded PDFs are stored as private MinIO objects, so they do not need to be embedded in the SQLite database.

Important deployment constraint

The Biashara OS Deployment uses strategy Recreate and one replica to avoid simultaneous access by multiple application processes to the same local SQLite database.

Ingress and application request flow

How a browser request reaches the Biashara OS service.

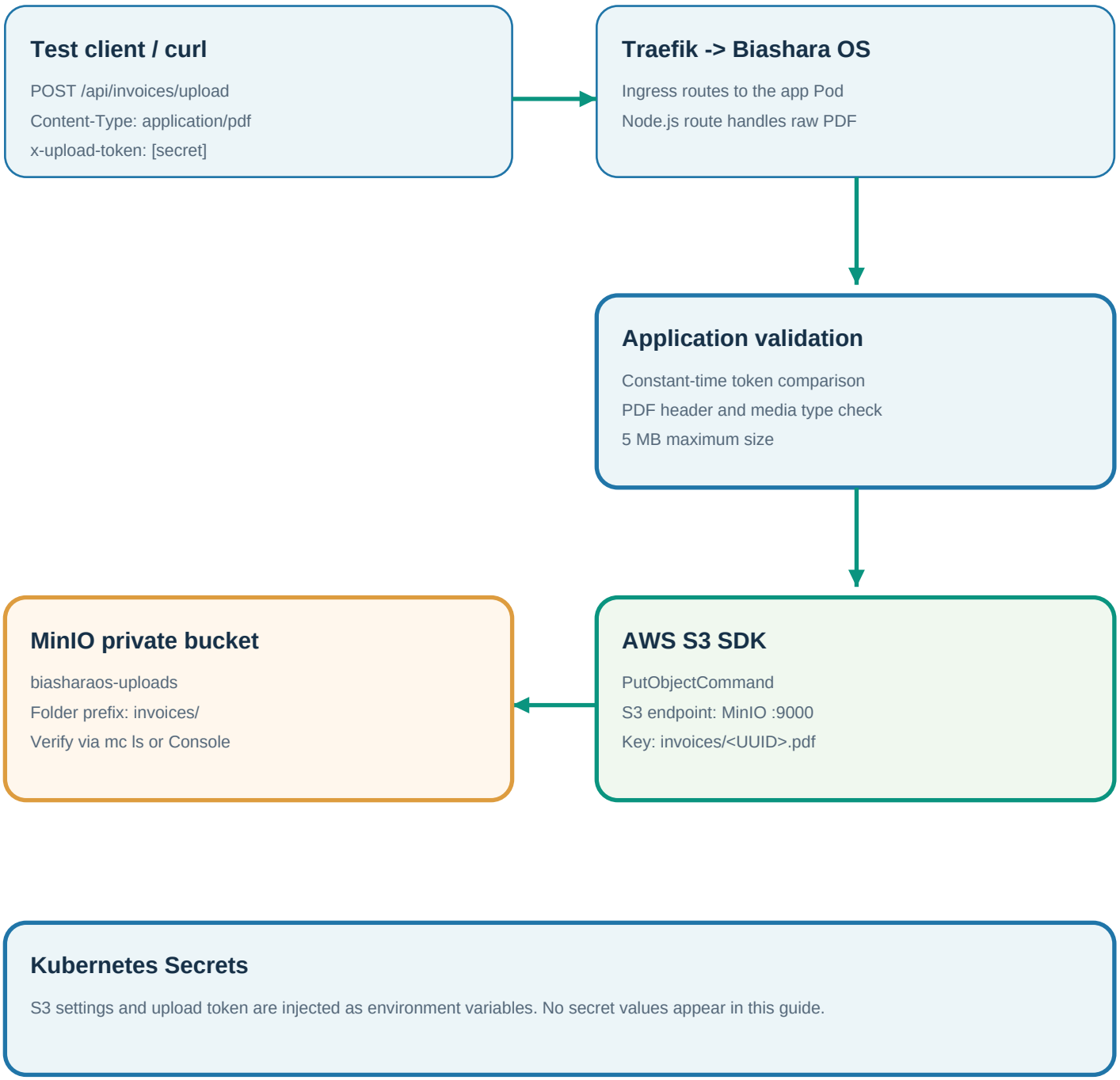


How we validated routing

curl -I -H "Host: biasharaos.home" http://192.168.30.134/ returned HTTP 200. The Host header is required for Traefik to select the expected hostname rule.

Protected PDF invoice upload flow

Test path from a PDF to a private S3-compatible object.



Result and current scope

A successful test returns HTTP 201 with the uploaded object key. This proves the backend storage path; a dashboard upload button, user authentication, invoice metadata and downloads are future enhancements.

1 — Build the VMs and plan network connectivity

Reconstructed setup example; based on recorded addresses. Create two Ubuntu VMs and one Debian LXC in Proxmox, assign their existing VLANs/IPs, ensure the control plane and worker can reach one another, and allow the worker to reach the MinIO API across VLANs. Details such as original VM provisioning clicks, firewall rule IDs and exact install commands are not preserved.

On each Kubernetes VM — identify addresses and routes

```
hostnamectl
ip -br addr
ip route
```

Why: Confirms which interface has the intended VLAN address and where off-subnet traffic is sent. A wrong default gateway or VLAN assignment prevents join or S3 access.

On deploy — test MinIO port reachability

```
curl -I --connect-timeout 5 http://192.168.30.147:9000/minio/health/live
```

Why: Separates worker-level network reachability from later Kubernetes Pod-level networking. MinIO health probes may return a short response without an HTML page.

On kubes — check connectivity to worker

```
ping -c 3 192.168.50.172
```

Why: Tests basic routed connectivity between the control-plane and worker segments; ICMP rules may affect this independent of Kubernetes.

Network design: Keep VLAN 30 (control plane and MinIO) distinct from VLAN 50 (worker), and use a non-overlapping Calico Pod CIDR, 10.244.0.0/16. Avoid creating any Calico IPPool that overlaps the physical 192.168.30.0/24 LAN.

2 — Prepare Ubuntu for kubeadm + containerd

Reconstructed setup example — fresh VMs only. These show a common kubeadm bootstrap path consistent with the resulting cluster. They are not recovered exact historical commands.

On kubes and deploy — kernel/network prerequisites

```
sudo swapoff -a
sudo modprobe overlay
sudo modprobe br_netfilter
cat <<EOF | sudo tee /etc/sysctl.d/99-kubernetes.conf
net.ipv4.ip_forward=1
net.bridge.bridge-nf-call-iptables=1
EOF
sudo sysctl --system
```

Why: Kubelet generally expects swap handling configured, and bridged packets plus forwarding must function for Pod networking. Permanent swap and runtime settings should be reviewed for a fresh deployment.

On kubes and deploy — verify container runtime

```
sudo systemctl status containerd
sudo ctr version
```

Why: The finished installation used containerd as Kubernetes CRI runtime. Verify it before launching kubeadm or troubleshooting Pods.

Fresh-install checklist: install containerd with systemd cgroups, kubelet/kubeadm/kubectl from the intended Kubernetes package repository, enable kubelet, and ensure the Kubernetes/containerd versions are compatible. The original apt install

lines and repository configuration were not retained, so this manual does not present invented package commands as fact.

3 — Initialize cluster and join the worker

Reconstructed setup example — never rerun on an existing cluster. The command structure below illustrates how the observed Pod CIDR would have been established.

On kubes — bootstrap shape

```
sudo kubeadm init --apiserver-advertise-address=192.168.30.134 \
  --pod-network-cidr=10.244.0.0/16
```

Why: Initializes a new control plane with the Pod CIDR later used by Calico. The exact original init arguments and Kubernetes version flags were not recorded.

On kubes — kubectl configuration

```
mkdir -p "$HOME/.kube"
sudo cp /etc/kubernetes/admin.conf "$HOME/.kube/config"
sudo chown "$(id -u):$(id -g)" "$HOME/.kube/config"
```

Why: Allows the operator to query and manage the new control plane; admin.conf has powerful permissions and must not be committed to GitHub.

On deploy: join using the unique token and discovery CA hash produced by kubeadm init (omitted for security). Your cluster subsequently showed worker **deploy** and control plane **kubes** with Kubernetes v1.36.x.

On kubes — verify both nodes

```
kubectl get nodes -o wide
kubectl get pods -A -o wide
```

Why: Confirms the worker joined and cluster system Pods are scheduled. These are recorded verification commands.

4 — Install Calico and diagnose Pod networking

Calico v3.28.0 was running with Pod network **10.244.0.0/16**, **IPIP Always** and outgoing NAT. The exact original Calico manifest/apply command is not preserved; installing a matching version on a fresh cluster requires following the corresponding Calico release instructions.

On kubes — inspect the pool and Pods

```
kubectl get ippools.crd.projectcalico.org -o wide
kubectl get pods -A -o wide
kubectl get pods -n kube-system -o wide
```

Why: These recorded commands exposed two pools: the intended Pod pool and an obsolete pool overlapping 192.168.30.0/24. The Pod address listing helped verify which addresses were actually in use.

Real incident: worker could reach MinIO; Pod could not

The worker host could reach **192.168.30.147:9000**, while the Biashara OS Pod initially could not. The stale **default-ipv4-ippool** overlapped the physical VLAN 30 subnet and affected Calico egress/NAT behavior. We verified workloads did not depend on it before removal.

On kubes — remove unused conflicting pool (incident-specific)

```
kubectl delete ippool default-ipv4-ippool
kubectl get ippools.crd.projectcalico.org
```

Why: After verifying the obsolete pool was unused, removal left new-ipv4-pool active. Pod-to-MinIO health requests then worked. Never blindly run this deletion on another cluster.

Observed outcome: Pod-to-MinIO connectivity recovered. The exact original in-Pod curl command and its response were not retained, so the guide does not fabricate them.

5 — Build Biashara OS v0.1.0 on the worker

The application repository was **github.com/Jamo642/biasharaos**. Source checkout, Node.js 24, and the original build were on **deploy**. The initial repository clone syntax below is a reconstruction; the subsequent image format and tags are recorded.

On deploy — fetch application (setup example)

```
git clone https://github.com/Jamo642/biasharaos.git
cd biasharaos
cat package.json
cat Dockerfile
```

Why: Makes the Node.js source available for container builds and inspects its entrypoint. The actual app used node server.js and port 3000.

On deploy — build and hand image to containerd

```
podman build -t localhost/biasharaos:0.1.0 .
podman save --format docker-archive -o /tmp/biasharaos-0.1.0.tar \
  localhost/biasharaos:0.1.0
sudo ctr -n k8s.io images import /tmp/biasharaos-0.1.0.tar
sudo ctr -n k8s.io images list | grep biasharaos
```

Why: Podman and containerd have separate stores. Export/import supplies the worker kubelet with the local image that uses imagePullPolicy Never. This workflow and 0.1.0 image were observed.

6 — Persist the SQLite database

The finished cluster has a local PV **biasharaos-sqlite-pv** bound to PVC **biasharaos-sqlite-pvc**. The worker directory **/var/lib/biasharaos/sqlite** is mounted at **/app/data**. The source uses **node:sqlite** and writes **/app/data/biashara.sqlite**.

On kubes — inspect persistent objects

```
kubectl get pv
kubectl get pvc -n biasharaos
kubectl describe pvc biasharaos-sqlite-pvc -n biasharaos
```

Why: Checks the claim is Bound and its mount is defined. Recreating or deleting this volume could destroy persistent business data.

On kubes — verify safe update strategy

```
kubectl get deployment biasharaos -n biasharaos \
  -o jsonpath='{.spec.strategy.type}{"\n"}'
```

Why: Shows Recreate. That strategy avoids old and new Node processes writing the same local SQLite file concurrently during image replacement.

Initial YAML: the original PV/PVC/Deployment manifest text was not retained. Its observed essentials were one replica on worker deploy, a container port of 3000, local imagePullPolicy Never, a PVC mount at /app/data, and Deployment strategy Recreate. Do not overwrite a live manifest from a generic example.

7 — Route traffic with Traefik Ingress

Traefik was installed in namespace **traefik**. Biashara OS exposes its app service on port 3000, with Ingress host **biasharaos.home** on HTTP port 80. The exact Helm install command and original Ingress YAML were not retained.

On kubes — inspect routing resources

```
kubectl get pods -n traefik -o wide
kubectl get ingress -n biasharaos
kubectl get services -n biasharaos
```

Why: Checks the proxy, the host-based routing rule, and the Service forwarding traffic toward Biashara OS.

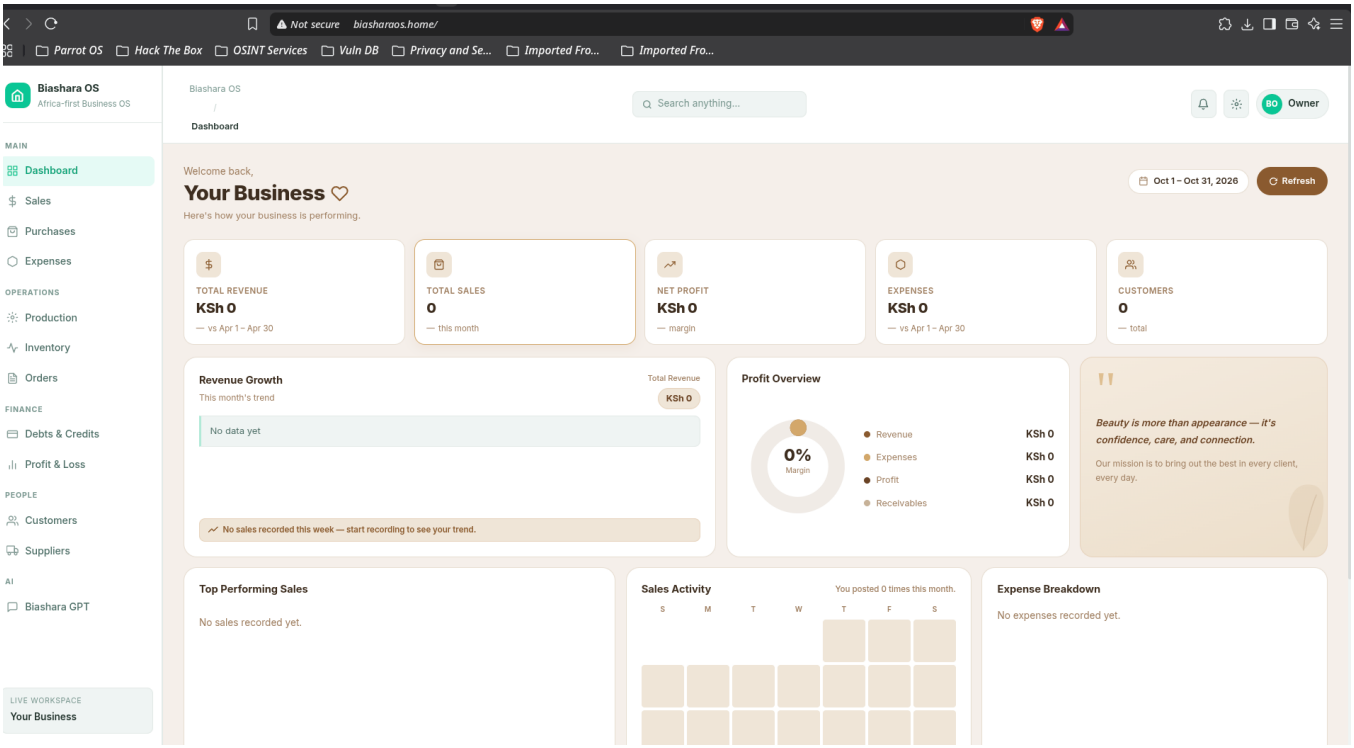
On kubes — prove routing without local DNS

```
curl -I -H 'Host: biasharaos.home' http://192.168.30.134/
```

Why: Sets the HTTP Host header so Traefik selects the right Ingress rule. The recorded response was HTTP 200, confirming ingress connectivity.

```
HTTP/1.1 200 OK  # observed status; response headers omitted
```

On the laptop: add 192.168.30.134 biasharaos.home to your hosts file or use LAN DNS. Opening **http://biasharaos.home** reached the running application. The browser showed Not secure because this lab route used HTTP rather than TLS.



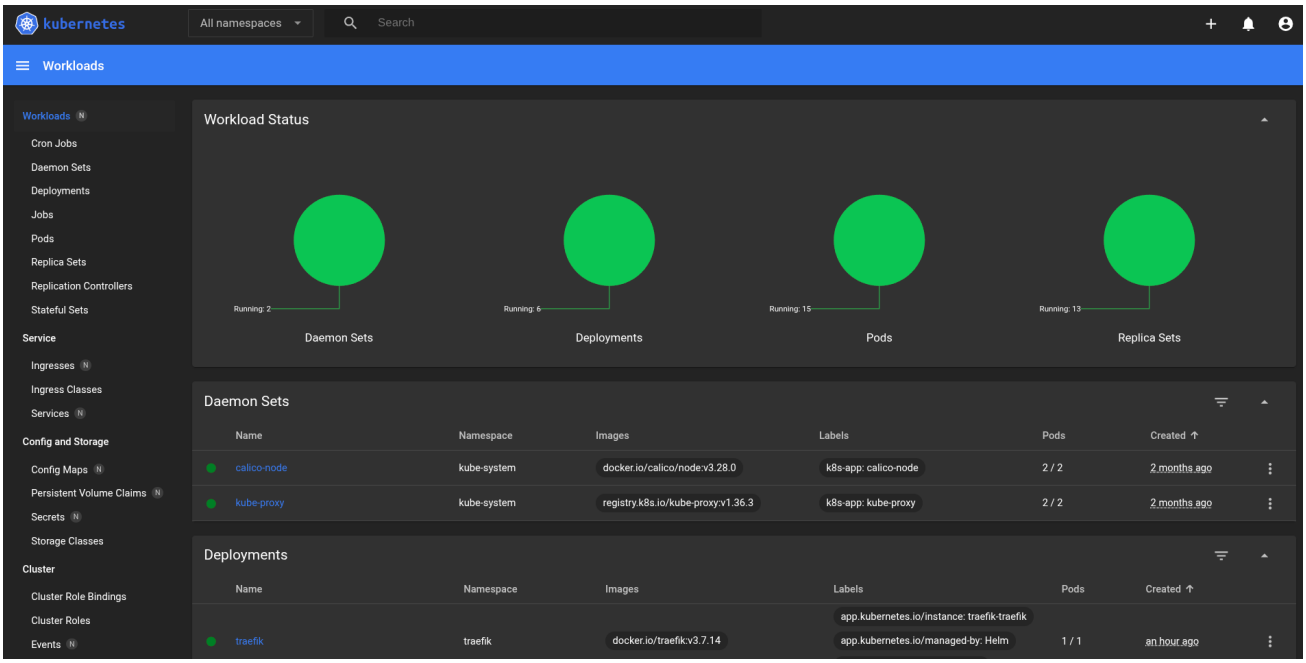
Evidence — Biashara OS dashboard loaded at biasharaos.home over Traefik Ingress.

8 — Verify the Kubernetes dashboard and application

On kubes — verify workloads

```
kubectl get deployments -A
kubectl get pods -n biasharaos -o wide
kubectl get deployment biasharaos -n biasharaos -o wide
```

Why: Checks both cluster-wide health and the application-specific Deployment, Pod readiness, and assigned worker. Kubernetes Dashboard is optional for these checks.



Evidence — Kubernetes Dashboard all-namespaces workload status.

The Dashboard screenshots are observation evidence. Kubernetes Dashboard authentication commands and login tokens are deliberately not included.

9 — Prepare MinIO storage outside Kubernetes

MinIO ran as an enabled Debian LXC systemd service on **s3bucket**. The S3 API was **192.168.30.147:9000**, and the browser console used port **9001**. Data was stored under **/var/lib/minio**. The installation/package and original service-file commands were not fully retained.

On s3bucket — confirm MinIO health

```
sudo systemctl status minio
sudo systemctl is-enabled minio
mc admin info local
```

Why: Checks service liveness, restart persistence, and authenticated access to the MinIO instance using the already-configured local mc alias.

On s3bucket — confirm bucket exists

```
mc ls local
mc ls local/biasharaos-uploads
```

Why: Shows the private bucket used for invoice storage. It was created and versioned, and a dedicated user biasharaos-app with a scoped policy was configured.

Permissions: the recorded policy intent was to allow listing/writing/reading objects within biasharaos-uploads, with no DeleteObject. The exact provisioning command sequence and credentials were not preserved. Do not publish

/etc/default/minio contents.

10 — Wire MinIO into Biashara OS

The application uses six Kubernetes Secret values: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, S3_ENDPOINT, S3_BUCKET, AWS_REGION, and S3_FORCE_PATH_STYLE. Sensitive values are excluded.

On kubes — inject existing Secret

```
kubectl get secret biasharaos-s3 -n biasharaos
kubectl set env deployment/biasharaos -n biasharaos \
  --from=secret/biasharaos-s3
kubectl rollout status deployment/biasharaos -n biasharaos --timeout=120s
```

Why: Adds the S3 connection settings to the Node Pod without baking secrets into images. This particular change triggered the stalled rollout diagnosed next.

11 — Debug stalled Pod shutdown caused by AppArmor

Actual incident: rollout status reported zero updated replicas and timed out. Kubernetes events reported FailedKillPod / KillContainerError / runc unable to signal init: permission denied. The old Node.js process stayed running as PID 7512.

On kubes — identify stalled rollout

```
kubectl get pods -n biasharaos -o wide
kubectl get events -n biasharaos --sort-by=.lastTimestamp
kubectl rollout status deployment/biasharaos -n biasharaos --timeout=120s
```

Why: The events and timeout directed attention to container termination rather than MinIO, application syntax, or the network.

On deploy — inspect containerd and old process

```
sudo systemctl status containerd
sudo ctr -n k8s.io containers list | grep biasharaos
sudo ctr -n k8s.io tasks list
ps -o pid,ppid,user,stat,cmd -p 7512
sudo kill -0 7512 && echo "Process exists"
```

Why: Confirmed containerd was active and the old application task still lived. kill -0 is only an existence check, not termination.

```
PID    PPID USER     STAT CMD
7512   7463 deploy   Ssl  node server.js
Process exists
```

On deploy — inspect AppArmor denials

```
sudo journalctl -k --since "20 minutes ago" --no-pager \
  | grep -iE 'apparmor|denied|audit|runc' | tail -30
sudo aa-status | grep -E 'cri-containerd|runc'
sudo cat /etc/apparmor.d/cri-containerd.apparmor.d
sudo cat /etc/apparmor.d/runc
```

Why: Kernel logs recorded denied TERM and KILL signal reception by the container profile. The policy contained MicroK8s-specific receive permissions but lacked a matching runc receive rule in this kubeadm environment.

```
apparmor="DENIED" operation="signal" class="signal"
profile="cri-containerd.apparmor.d" comm="runc"
denied_mask="receive" signal=term peer="runc"
```

On deploy — back up and fix specific profile

```
sudo cp /etc/apparmor.d/cri-containerd.apparmor.d \
  /etc/apparmor.d/cri-containerd.apparmor.d.bak
sudo sed -i '/signal (receive) peer=snap.microk8s.daemon-containerd,/a\
  signal (receive) peer=runc,' /etc/apparmor.d/cri-containerd.apparmor.d
sudo apparmor_parser -r /etc/apparmor.d/cri-containerd.apparmor.d
```

Why: Added the narrowly scoped receive permission for runc and reloaded only this profile. Avoided disabling AppArmor globally and avoided force-deleting a SQLite writer.

On kubes — confirm recovery

```
kubectl rollout status deployment/biasharaos -n biasharaos --timeout=120s
```

Why: Kubernetes could now stop the previous Pod and complete replacement.

```
deployment "biasharaos" successfully rolled out
```

12 — Extend Node.js 24 for authenticated invoice uploads

Initial code used **node:http**, **node:sqlite**, a JSON/urlencoded parser, and had no native PDF upload endpoint or inbound API login. We inspected the source before changing it to avoid breaking existing routes.

On deploy — inspect backend and build behavior

```
cd ~/biasharaos
cat package.json
cat Dockerfile
sed -n '1,85p' server.js
sed -n '2920,2980p' server.js
grep -niE 'authorization|session|jwt|bearer|login' server.js | head -60
```

Why: Revealed where request parsing occurs, confirmed no obvious existing upload authentication, and showed that the original Dockerfile did not install npm dependencies.

On deploy — install official S3 SDK

```
npm install @aws-sdk/client-s3
npm ls @aws-sdk/client-s3
```

Why: Adds the S3-compatible JavaScript client that can issue PutObject requests to MinIO.

Dockerfile change (recorded source edit)

```
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
```

Why: Ensures production dependencies exist inside the built Node.js 24 image rather than only in the worker checkout.

Backend change (recorded design)

```
const { S3Client, PutObjectCommand } = require("@aws-sdk/client-s3");
// client configured from S3_ENDPOINT, AWS_REGION, AWS_ACCESS_KEY_ID,
// AWS_SECRET_ACCESS_KEY, S3_BUCKET; forcePathStyle: true
if (req.method === "POST" && url.pathname === "/api/invoices/upload") {
  return await uploadInvoice(req, res);
}
```

The handler was inserted **before** await parseBody(req) because the existing parser expects JSON/URL-encoded data and cannot parse a raw PDF. The upload reads up to 5 MiB, checks application/pdf plus the %PDF- signature, compares a separate API upload token, and writes a unique invoices/<UUID>.pdf object to the private bucket.

On deploy — validate edits safely

```
node --check server.js
grep -nE 'readInvoicePdf|uploadInvoice|PutObjectCommand' server.js
grep -nE '/api/invoices/upload|parseBody\(req\)' server.js
```

Why: No output from node --check means syntax passed. Repeated small checks prevented a pasted duplicate import from being saved into the final version.

13 — Configure a separate upload authentication token

On kubes — create and connect dedicated API token

```
kubectl create secret generic biasharaos-invoice-token -n biasharaos \
  --from-literal=INVOICE_UPLOAD_TOKEN="$(openssl rand -hex 32)"
kubectl set env deployment/biasharaos -n biasharaos \
  --from=secret/biasharaos-invoice-token
```

Why: Creates a random API upload token distinct from MinIO keys and Kubernetes Dashboard credentials. Secrets stay outside version control. This causes a Deployment rollout.

On kubes — check presence, not value

```
kubectl exec -n biasharaos deployment/biasharaos -- \
  node -e 'console.log(process.env.INVOICE_UPLOAD_TOKEN ? "configured" : "missing")'
```

Why: Verifies the running container received the token without exposing it.

14 — Build and roll out Biashara OS v0.2.0

On deploy — build and load updated app image

```
cd ~/biasharaos
node --check server.js
podman build -t localhost/biasharaos:0.2.0 .
podman save --format docker-archive -o /tmp/biasharaos-0.2.0.tar \
  localhost/biasharaos:0.2.0
sudo ctr -n k8s.io images import /tmp/biasharaos-0.2.0.tar
sudo ctr -n k8s.io images list | grep biasharaos
```

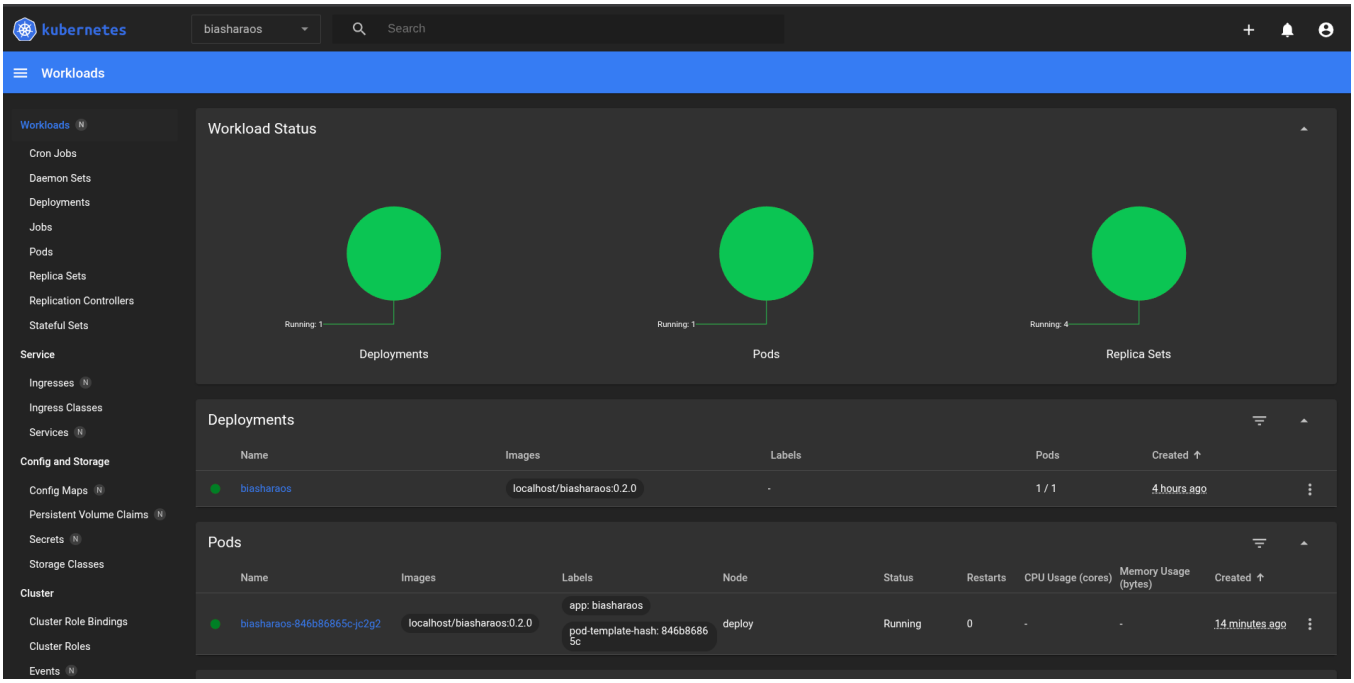
Why: Builds a versioned image with the S3 SDK, then imports it into the worker containerd store that kubelet uses.

On kubes — safe image rollout

```
kubectl get deployment biasharaos -n biasharaos \
  -o jsonpath='{.spec.strategy.type}{"\n"}'
kubectl set image deployment/biasharaos \
  biasharaos=localhost/biasharaos:0.2.0 -n biasharaos
kubectl rollout status deployment/biasharaos -n biasharaos --timeout=180s
kubectl get pods -n biasharaos -o wide
kubectl logs -n biasharaos deployment/biasharaos --tail=30
```

Why: Recheck Recreate for SQLite safety, replace the image, wait for readiness and inspect startup logs.

```
Biashara OS running on http://localhost:3000
```



Evidence — namespace biasharaos: v0.2.0 Deployment and its ready Pod on node deploy.

15 — Upload a test PDF into private MinIO

The user generated a small sample invoice on kubes, then used a curl POST with raw PDF bytes and an API token. Actual token bytes and the full successful HTTP response were not pasted into the transcript.

On kubes — locate or create a PDF

```
find ~ -maxdepth 2 -iname '*.pdf' | head
```

Why: Looks for an available PDF. When none was present, a short Python script generated /tmp/test-invoice.pdf.

On kubes — send invoice to Traefik/API

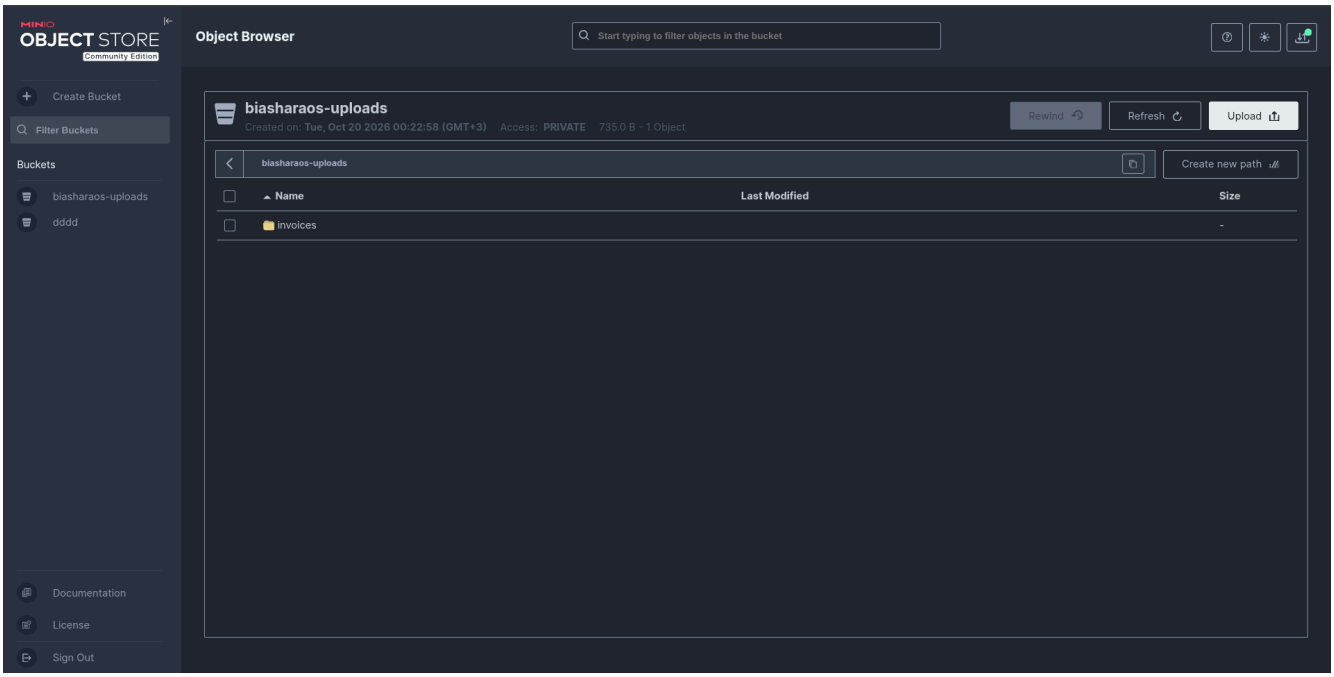
```
TOKEN=$(kubectl get secret biasharaos-invoice-token -n biasharaos \
  -o jsonpath='{.data.INVOICE_UPLOAD_TOKEN}' | base64 -d)
curl -i -X POST http://192.168.30.134/api/invoices/upload \
  -H 'Host: biasharaos.home' \
  -H "x-upload-token: $TOKEN" \
  -H 'Content-Type: application/pdf' \
  --data-binary @/tmp/test-invoice.pdf
unset TOKEN
```

Why: Routes through the same Ingress host, verifies the dedicated upload token and streams the PDF bytes into the MinIO S3 bucket. A successful request should return HTTP 201.

On s3bucket — independently confirm object

```
mc ls local/biasharaos-uploads/invoices/
```

Why: Displays invoices stored in the private bucket. The user confirmed the upload succeeded and supplied the MinIO console screenshot showing one object under invoices.



Evidence — private biasharaos-uploads bucket with invoices prefix; 1 object shown.

16 — What we completed, and what remains

Validated in the lab: control-plane/worker deployment; Traefik host routing; SQLite mounted on persistent storage; corrected Calico overlap; recovered AppArmor-blocked rollout; Node.js v0.2.0 starts; protected test-PDF upload reaches MinIO.

Not yet claimed: a full dashboard upload UI, invoice-to-transaction metadata linking, comprehensive role-based authentication, TLS, off-host SQLite backups, highly available storage, or production-grade monitoring/restore testing. Those are sensible follow-on changes, not parts of the verified test.

Historical accuracy note: Initial Proxmox VM provisioning clicks, exact kubeadm/containerd/Calico/Traefik installation lines, initial Kubernetes YAML, and the precise MinIO policy creation commands were not fully available. They have been distinguished from observed commands; no secret values or Kubernetes Dashboard login-token generation steps appear here.